

## 03：第一版为什么改成 NBFNet 第二版

---

### 1. 为什么项目会引入机器学习

---

传统区域策略很容易说明：

- 随机区域：随机选种子，再用 BFS 生长；
- OD 热点区域：从历史中经常作为起点或终点的节点附近生长。

但“端点很热”不等于“适合被压缩”。一个区域是否有价值，还取决于未来查询是否会从它附近经过、穿过它能减少多少搜索、边界有多复杂、端点接入是否昂贵，以及它占用多少索引预算。

这些因素互相作用，单一手工规则很难完整表达，所以项目尝试让模型从道路结构和历史 OD 中学习区域价值。

### 2. GNN 是什么

---

GNN 是 Graph Neural Network，中文叫图神经网络。普通神经网络通常把每个样本看成一行独立数值；GNN 还允许一个节点读取邻居节点的信息。

一层典型消息传递可以粗略理解为：

节点自己的特征 + 邻居传来的摘要 → 节点的新表示

堆叠多层后，一个节点可以间接吸收更远邻居的信息。

项目使用 GNN 的初衷不是让它重做 Dijkstra，而是让它理解：历史需求如何沿道路结构影响某个节点或某个区域的压缩价值。

### 3. 第一版做了什么

第一版使用两层 GraphSAGE 为原图节点打分。GraphSAGE 是一种 GNN：它聚合邻居表示，再与节点自身表示组合，得到下一层节点表示。

第一版大致流程是：

历史 OD 和道路节点特征



构造节点级代理目标 Proxy



GraphSAGE 学节点种子价值



从高分节点生长 512 节点区域



再用端点风险惩罚修正区域排序



选不重叠区域并构建精确压缩索引

第一版代码与证据已经归档到 [archive/gnn\\_v1/](#)，不再是当前主线入口。

### 4. Proxy 是什么

Proxy 可以译为“代理指标”或“替代目标”。当真正想优化的东西太贵、暂时拿不到，研究者会先构造一个容易计算、希望与真实目标相关的量。

第一版 Proxy 的核心直觉是：

1. 取每条历史 OD 起点和终点经纬度的几何中点；
2. 把这些中点的密度扩散到附近道路节点；
3. 同时估计起终点需求扩散形成的端点风险；
4. 中点密度越高，节点可能越接近常见 OD 的中间地带；
5. 端点风险越高，区域内部端点回退问题越严重；

6. 用“中点价值除以端点风险惩罚”构造节点目标。

它不使用真实历史最短路径，因此仍满足“无路径监督”，但它只是人设计的几何近似，不是区域压缩后真实减少了多少查询工作量。

## 5. 第一版最初看起来是成功的

---

第一版完整 GNN 在相近 shortcut 预算下，五个随机种子的平均在线耗时下降

14.35%  $\pm$  0.62%，随机区域是 9.78%  $\pm$  1.37%。所有查询正确率都是 100%。

单看这个结果，很容易得出“GNN 有效”的结论。但严谨研究不能只比较完整系统与随机方法，还必须回答：完整系统中的哪一部分真正产生了收益？

## 6. 什么是消融实验

---

消融实验是指从完整方法中删除或替换某个组件，再观察性能变化。例如：

- 把 GraphSAGE 换成不读取邻居的 MLP；
- 完全不用学习模型，只用 Proxy 排序；
- 只保留风险项；
- 删除端点风险惩罚；
- 改变固定扩散步数。

如果删除某组件后性能几乎不变，就不能声称该组件有重要独立贡献。

## 7. 第一版消融揭示了什么

---

关键结果包括：

| 方法             | 平均耗时变化          | 解释            |
|----------------|-----------------|---------------|
| 完整第一版 GNN，5 种子 | -14.35% ± 0.62% | 看起来很好         |
| MLP，5 种子       | -13.93% ± 0.75% | 不做图消息传递也很接近   |
| Proxy-only     | -14.05%         | 甚至不训练模型也很接近   |
| Risk-only      | -7.86%          | 风险本身贡献明显但不够完整 |
| 不使用端点风险的 GNN   | -1.68% ± 0.29%  | 性能大幅下降        |

这说明：

- 1. 历史 OD 确实能指导区域选择；
- 2. 但第一版 GraphSAGE 相对 MLP 的独立贡献没有成立；
- 3. 第一版的大部分收益能被手工几何 Proxy 和端点风险解释；
- 4. “完整 GNN 优于随机”不等于“图传播本身有效”；
- 5. 继续拟合同一个 Proxy，只会让模型复制手工规则，不能验证最初的传播假设。

## 8. 为什么端点风险会压倒其他信号

第一版压缩索引删除区域内部节点。如果查询起点或终点落在内部节点，旧查询器会整条查询回退原图。

于是高频端点附近的区域即使非常适合被许多路线穿过，也可能因为大量查询端点落在内部而频繁回退。模型最容易学到的捷径变成：

不要压缩高频端点附近。

这不是完全错误的规律，但它掩盖了项目真正想研究的规律：起点需求和终点需求如何沿有向

道路传播，并在潜在通行走廊中交汇。

## 9. 固定无向扩散为什么也不够

---

第一版把需求做固定、无向、均值式扩散：

- 固定：扩散规则由人写死，不从真实收益学习；
- 无向：忽略单行道和道路方向；
- 均值：邻居信息按简单平均聚合；
- 固定步数：传播多远由超参数决定。

步数消融没有显示稳定增益，只能说明“这套简单扩散”没有显著价值，不能推出“任何有向、可学习、区分起点和终点的传播都无效”。

## 10. 第二版的六个纠偏决定

---

第二版因此冻结了以下方向：

1. **先修查询器**：内部端点不再整图回退，消除端点风险对标签的结构性支配；
2. **固定候选池**：所有模型面对完全相同的 1,200 个区域，不把候选变化误当模型贡献；
3. **使用真实收益标签**：直接测量一个区域让未来查询少展开多少节点；
4. **分开起点与终点**：起点需求沿原图正向传播，终点需求沿反图传播；
5. **学习传播规则**：边权、门控和传播深度由真实收益损失反向更新；
6. **更严格对照**：必须优于无传播 MLP、固定扩散、无向传播和 cost-only，而不只是优于随机。

## 11. 第二版要证明的核心命题

---

第二版不把“模型预测准确”当作唯一目标。它要回答一个更具体的问题：

在只使用过去 OD 起终点与静态有向道路图、不读取历史最短路径的条件下，可学习的双向

需求传播是否能稳定发现比原始频率、固定扩散和成本特征更有价值的压缩区域？

如果最终完整模型只和无传播 MLP 一样好，就不能说 GNN 的道路传播贡献成立。负结果也必须保留。

## 12. 为什么第二版选择 NBFNet

第一版的主要问题不是“层数不够”，而是传播没有真正受 OD 查询条件控制。GraphSAGE 把每个节点放进同一个全图传播过程，固定扩散又是无向、均值和固定步数；模型很容易只复制端点频率和几何 Proxy。

NBFNet 提供了更合适的骨架：从需求源注入状态，沿有向边逐步传播，消息可以读取道路属性，并保留不同传播深度的信息。第二版不会逐条 OD 在全图上暴力运行原始 NBFNet，而是把历史 OD 聚合成少量保持起终点配对的需求原型，再进行分批条件传播。这样保留查询条件化思想，又能适应 Porto 图规模。

第二版的正式名称是：

### OD 条件化双向 Neural Bellman-Ford 区域价值网络

它预测的是固定候选区域的未来真实压缩收益，不预测路径、不替代 Dijkstra，也不改变 100% 精确距离保证。

## 13. 版本时间线

仓库历史和当前工作区显示的主要节点是：

| 日期         | 事件                         |
|------------|----------------------------|
| 2026-07-13 | 整理第二版实施计划，开始把真实区域收益作为主监督方向 |
| 2026-07-14 | 冻结真实收益驱动的双向需求场研究边界         |

| 日期         | 事件                                                           |
|------------|--------------------------------------------------------------|
| 2026-07-15 | 完成精确端点局部接入并归档第一版 GNN 与旧计划                                    |
| 2026-07-22 | 合并相关分支，主线保留第二版查询基础设施                                         |
| 2026-07-27 | 完成旧候选池的标签链路与正式 $1,200 \times 2,000$ 标签                       |
| 2026-07-29 | 正式确定 OD 条件化双向 NBFNet；候选改为全图固定随机并重算标签                         |
| 2026-07-30 | 完成新候选正式标签、统一数据和五种子 CUDA MLP，状态 <code>ready_for_nbfnet</code> |

下一章解释第二版为了防止信息泄漏和“模型走捷径”，如何把整个系统严格分成四层。